

String Methods

methods — Actions that can be performed on objects

Format : `stringName.method()`

1. Trim Method

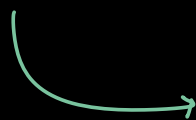
— whitespaces from both ends of string & returns a new one.

```
let msg = "   Hi   ";  
msg.trim();
```

o/p: 'Hi'

msg

o/p: ' Hi '



what really happens?

trim doesn't modify the string itself.
trim uses the string to create a new string and remove spaces from there.

Strings are IMMUTABLE in JS

- No changes can be made to strings
- Whenever a change is made, a new string is created and the old string remains same.

2. toUpperCase & toLowerCase

```
let str = "Random string";
```

```
str.toUpperCase()
```

```
"RANDOM STRING"
```

```
str.toLowerCase()
```

```
"random string"
```

3. indexOf

- returns first index of occurrence of some value in string.

```
let str = "I Love Coding";
```

```
str. index Of ( " love " ) // 1
str. index Of ( " J " ) // -1 (not found)
str. index Of ( " o " ) // 2 (only 1st index)
```

Method Chaining

- using one method after another.
- Order of execution : left to Right

Ex: str. to Upper Case(). trim()

4. slice

- Returns a part of the original string as a new string

- o `str.slice ( start , end )`
1 5 → end not included
ie, index 1 to index 4
- o `str.slice ( start )`
5 → end is assumed to be str.length
- o `str.slice ( -ve num )` → converts it to str.length - num
-2

```
let str = " I Love Coding " ;
```

str.slice(5) → " Coding "

str.slice(1, 5) → " love "

str.slice(-2) → " ng "

4. replace

- searches a value of the string and returns a new string with the value replaced

```
let str = " I Love Coding " ;
```

str.replace("love", "do") → " I do Coding "

str.replace("o", "x") → " I l x ve Coding "

↳ only 1st occurrence is replaced.

5. repeat

— Returns a string with the number of copies of a string.

```
let str = "Mango"
```

```
str.repeat(3);
```

→ "Mango Mango Mango"

ARRAY

— Arrays are mutable

```
let fruits = ["mango", "apple", "litchi"] ;  
fruits[0] = "banana" ;  
fruits ;
```

o/p → (3) ['banana', 'apple', 'litchi']

Array method

- push : add to end
- unshift : add to start
- pop : delete from end and returns it
- shift : delete from start and returns it

◦ `indexOf()` → returns index
↳ If not found, returns -1

◦ `includes()` → search for a value
→ found: `true`
not found: `false`

◦ `concat()` `let num1 = ["1", "2", "3"];`
 `let num2 = ["4", "5"];`
 `num1.concat(num2);`
→ (6) ['1', '2', '3', '4', '5']

◦ `reverse()` `num1.reverse();`
→ (3) ['3', '2', '1']

◦ `slice()` (gives a copy)

- Returns a part of the original string as a new string
Ex: ["0", "1", "2", "3", "4"]

- `num1.slice(start, end)`
 1 3 → end not included
 ie, index 1 to index 2
→ (2) ['1', '2']

- `num1.slice(start)`
 1 → end is assumed to be str.length
→ (4) ['1', '2', '3', '4']

- `num1.slice(-ve num)` → converts it to $(\text{str.length} - \text{num})$
 -2
→ (2) ['3', '4']

- `splice()` : removes / replaces / add elements in place
(makes changes to original string)

`splice(start, deleteCount, item 0, .... item N)`
↳ optional

`let colors = ["red", "yellow", "blue", "orange", "pink", "white"];`

```
> colors.splice(4);
< ▶ (2) ['pink', 'white']
> colors
< ▶ (4) ['red', 'yellow', 'blue', 'orange']
> colors.splice(0, 1);
< ▶ ['red']
> colors
< ▶ (3) ['yellow', 'blue', 'orange']
> colors.splice(0, 1, "black", "grey");
< ▶ ['yellow']
> colors
< ▶ (4) ['black', 'grey', 'blue', 'orange']
```

Note : `let cars = ['toyota', 'xuv', 'maruti', 'bmw', 'ferrari'];`

```
> cars
< ▶ (5) ['toyota', 'xuv', 'maruti', 'bmw', 'ferrari']
> cars.splice(1, 0, "mercedes");
< ▶ []
> cars
< ▶ (6) ['toyota', 'mercedes', 'xuv', 'maruti', 'bmw', 'ferrari']
> cars.splice(1, 1, "gwagon");
< ▶ ['mercedes']
> cars
< ▶ (6) ['toyota', 'gwagon', 'xuv', 'maruti', 'bmw', 'ferrari']
>
```

→ add mercedes
to index 1

→ removes mercedes
from index 1 and
add gwagon

◦ `sort()` : sorts an array

```
> let chars = ['b', 'd', 'e', 'a'];  
< undefined  
> chars.sort();  
< ▶ (4) ['a', 'b', 'd', 'e']  
> let marks = [99, 89, 67, 42, 100];  
< undefined  
> marks.sort();  
< ▶ (5) [100, 42, 67, 89, 99]
```

→ sorts based on alphabets

→ converts no. to unicode and then sorts them

Constant Arrays

`const arr = [1, 2, 3, 4];`



→ These no. are not made const.

arr is made constant.

◦ arr stores addresses and not numbers.

Nested Arrays

array of arrays

`let nums = [[2, 4], [3, 6], [4, 8]];`

o/p : (3) [array(2), array(2), array(2)]

(2) [2, 4]

(2) [3, 6]

(2) [4, 8]

For accessing: `nums[row][col]`

	0,0	0,1	
	2	4	
1,0	3	6	1,1
2,0	4	8	2,1

Loops with Nested Arrays

```
1 let heroes = [  
2   ["ironman", "spiderman", "thor"],  
3   ["superman", "wonder woman", "flash"]  
4 ]  
5  
6 for(let i=0; i<heroes.length; i++) {  
7   console.log(i, heroes[i], heroes[i].length);  
8   for(let j=0; j<heroes[i].length; j++) {  
9     console.log(`j=${j}, ${heroes[i][j]}`) }  
10 }  
11 }
```

1, 2

ironman, spiderman, ...

o/p:

(3) { 'ironman', 'spiderman', 'thor' }

j = 0 , ironman

j = 1 , spiderman

j = 2 , thor

(3) { 'superman', 'wonder woman', 'flash' }

j = 0 , superman

j = 1 , wonder woman

j = 2 , flash

for of Loop

```
let fruits = [ "mango", "apple", "banana", "litchi", "orange" ];  
for ( fruit of fruits ) {  
  console.log(fruit);  
}
```

works like 'i' in normal for loop

